JavaOne 2026 Workshop

LangChain4j-CDI

Generative AI Natively Injectable in Jakarta EE — Build production-ready AI agents using CDI, MicroProfile, and LangChain4j on WildFly.

Yann Blazart
Tech Lead @ SCIAM

Emmanuel Hugonnet
WildFly Core @ IBM

What You Will Build

In this hands-on workshop, you will progressively build three AI-powered applications using **LangChain4j-CDI**, turning Jakarta EE interfaces into injectable AI agents with a single annotation.

Exercise 1: AI Agent

Create an injectable AI chatbot with `@RegisterAIService`, streaming responses, and multimodal vision.

Exercise 2: Resilience & Observability

Add MicroProfile Fault Tolerance (`@Retry`, `@Timeout`, `@CircuitBreaker`) and OpenTelemetry.

🔌

Exercise 3: MCP Integration

Connect your AI agent to external tools via the Model Context Protocol (MCP) — "JDBC for AI".

💡 **WORKSHOP FORMAT**

Each exercise has a `base/` module (skeleton with TODOs) and a `solution/` module (complete reference). Work in `base/` and follow the steps. If you get stuck, the `solution/` is always there as a safety net.

Requirements

Software Prerequisites

TOOL	VERSION	PURPOSE
JDK	21+	Java development
Maven	3.9+	Build tool
IDE	Apache NetBeans / IntelliJ IDEA / VS Code	Code editor
curl	any	API testing
Mistral AI API Key	—	LLM provider (free tier available)
Docker / Podman	any	Grafana LGTM stack (Exercise 2 only)

1. Get the Source Code

```
git clone https://github.com/yblazart/confs-langchain4j-cdi-javaone2026.git
cd confs-langchain4j-cdi-javaone2026/demo-project
```

2. Set Up Mistral AI

Sign up at console.mistral.ai and follow these steps to get a free API key:

Step 1: Access API Keys

From the Mistral AI Studio homepage, click on your profile icon (top-right corner), then select **API Keys**.

Mistral AI Studio - Access API Keys from profile menu

Step 2: Choose a Plan

On the API Keys page, you will see a warning: *"No plan is currently active on this organization, the API keys will not be active."* Click **Choose a plan** to activate your keys.

Mistral AI - API Keys page with plan warning

> ⚠️ **IMPORTANT**
>
> Without an active plan, your API keys will be created but **will not work**. You will get an `Unauthorized` error when calling the API.

Step 3: Select the Free "Experiment" Plan

Choose the **Experiment** plan (left side) — it is completely **free** and gives you access to all frontier AI models. Click **"Experiment for free"**.

Mistral AI - Choose the free Experiment plan

Step 4: Accept Terms and Subscribe

Check the box to accept the Terms of Service and Privacy Policy, then click **Subscribe**.

Mistral AI - Sign up to Experiment plan

You will then be asked to provide your **phone number** to receive an **SMS activation code**. This is required to activate the free Experiment plan.

Step 5: Create and Export Your API Key

Once your plan is activated, go back to the API Keys page and click **"Create new key"**. Copy the key immediately — you won't be able to see it again.

Mistral AI - API key successfully created

Then export it in your terminal:

```
# Linux / macOS
export MISTRAL_API_KEY=your-api-key-here

# Windows (PowerShell)
$env:MISTRAL_API_KEY="your-api-key-here"
```

> 💡 **ALTERNATIVE: LOCAL OLLAMA**
>
> If you prefer running models locally, install Ollama and pull a model: `ollama pull qwen2.5:7b`. You will need to adjust the `microprofile-config.properties` to use OllamaChatModel instead of MistralAiChatModel.

3. WildFly (Automatic)

WildFly 39 is automatically downloaded and provisioned by the Maven plugin. No manual installation needed:

```
# This downloads WildFly, starts it, and deploys the app
cd demo-1-ai-agent/solution
mvn clean wildfly:dev
```

The `wildfly:dev` goal is your best friend — it hot-reloads on every `mvn package`.

4. Verify Your Setup

```
# Build all modules to download dependencies
cd demo-project
mvn clean install -DskipTests

# Quick smoke test
cd demo-1-ai-agent/solution
mvn clean wildfly:dev

# In another terminal:
curl -X POST -H "Content-Type: text/plain" \
  -d "Hello, tell me a joke" \
  http://localhost:8080/demo-1/api/chat
```

✓ READY!

If you get a response from the AI comedian, your environment is set up correctly. Stop the server (`Ctrl+C`) and proceed to the exercises.

Architecture Overview

LangChain4j-CDI is a CDI extension that bridges **LangChain4j** (the Java AI framework) with **Jakarta EE / MicroProfile** servers. It turns annotated interfaces into injectable AI agents.

How It Works

1. Scan
The CDI extension scans for `@RegisterAIService` interfaces at startup

2. Read attributes
For each interface: `chatModelName`, `tools`, `chatMemoryProviderName`, `contentRetrieverName`

3. Create synthetic bean
A LangChain4j proxy is wrapped in a managed CDI bean

4. Resolve components via SPI

The `LLMConfig` SPI reads from MicroProfile Config: `dev.langchain4j.cdi.plugin.<name>.class=...`

5. Inject!

`@Inject Assistant assistant;` — it just works

The Before / After

Without CDI (verbose)

```
ChatLanguageModel model =
  MistralAiChatModel.builder()
    .apiKey(System.getenv("MISTRAL_API_KEY"))
    .modelName("pixtral-large-latest")
    .temperature(0.3)
    .maxTokens(1000)
    .build();

ChatMemory memory =
  MessageWindowChatMemory.builder()
    .maxMessages(20).build();

Assistant assistant =
  AiServices.builder(Assistant.class)
    .chatLanguageModel(model)
    .chatMemory(memory)
    .tools(new BookingTools())
    .build();
```

With CDI (clean)

```
@RegisterAIService(
  chatModelName = "my-model",
  tools = BookingTools.class)
public interface Assistant {

  @SystemMessage("You are a
    booking assistant")
  String chat(
    @UserMessage String message);
}

// In your REST endpoint:
@Inject
Assistant assistant;
```

Dual CDI Extension Support

LangChain4j-CDI provides **both** extension types, so it works on **any** CDI server:

EXTENSION TYPE	PROCESSING	SERVERS
Portable Extension	Runtime	WildFly, Payara, GlassFish, Liberty
Build Compatible Extension	Compile-time	Quarkus, Helidon

Project Structure

```
demo-project/
├── pom.xml ← Parent POM (centralized versions)
├── demo-1-ai-agent/ ← Injectable AI Agent
│   ├── base/ ← Skeleton with TODOs (your workspace)
│   └── solution/ ← Complete reference
├── demo-2-ft-telemetry/ ← Memory + RAG + Tools + FT + Telemetry
│   ├── base/
│   └── solution/
└── demo-3-mcp/ ← MCP Integration
    ├── mcp-server/ ← Standalone MCP server (fat JAR)
    ├── base/
    └── solution/
```

Presentation Slides

The workshop includes a Reveal.js slide deck covering the theory behind each exercise. Here is what each section covers.

> 🎬 **LAUNCH SLIDES**
>
> From the project root: `cd slides && python3 -m http.server 8000`, then open http://localhost:8000. Press S for speaker notes, F for fullscreen, 0 for overview.

Introduction & Context

The talk opens with the problem statement: LangChain4j works great with Quarkus, but enterprises run on **WildFly, Payara, Liberty, GlassFish, Helidon**. The answer: `@RegisterAIService` — "Inject an AI agent as naturally as an EntityManager".

CDI Extension Internals

Deep dive into how the CDI extension scans `@RegisterAIService`, creates synthetic beans, resolves components via the `LLMConfig` SPI, and bridges to MicroProfile Config. Covers both Portable and Build Compatible extensions.

Progressive Features

- **Conversation Memory**: `@MemoryId` + `ChatMemoryProvider` for per-session state
- **RAG**: `contentRetrieverName` with embeddings and vector stores
- **Tools / Function Calling**: CDI beans with `@Tool` methods — the killer feature
- **MCP**: Model Context Protocol via `toolProviderName` — "JDBC for AI"

Production Ready

- **MicroProfile Fault Tolerance**: `@Retry`, `@Timeout`, `@Fallback`, `@CircuitBreaker`
- **MicroProfile Telemetry**: OpenTelemetry traces and metrics with Grafana LGTM
- **LangChain4j-CDI Modules**: core, portable-ext, build-compatible-ext, fault-tolerance, telemetry
- **A2A (Agent-to-Agent)**: preview of the next protocol for inter-agent communication

Exercise 1: Injectable AI Agent

Key message: "From 15 lines of boilerplate to 1 annotation + 1 config property"

> 📌 **GOAL**
>
> Create a simple AI chatbot using `@RegisterAIService`, inject it into a JAX-RS endpoint, and test it. Then add streaming and multimodal vision capabilities.

```
cd demo-project/demo-1-ai-agent/base
# Open this directory in your IDE
```

Key Files

FILE	PURPOSE
`ChatAssistant.java`	AI service interface (your main task)
`ChatAssistantStreaming.java`	Streaming AI service
`ChatResource.java`	JAX-RS REST endpoint
`ImageAnalyzerServlet.java`	Multimodal vision servlet
`microprofile-config.properties`	LLM configuration

1 Create the AI Service Interface

Open `ChatAssistant.java`. This is a plain Java interface. Your task: turn it into an injectable AI agent.

Add @RegisterAIService

```
@RegisterAIService(chatModelName = "my-model")
public interface ChatAssistant {

    @SystemMessage("""
        You are a Vegas stand-up comedian performing at a casino lounge.
        You tell jokes about tourists, slot machines, poker faces,
        and the glamorous chaos of Las Vegas. Your humor is sharp,
        self-deprecating, and full of Vegas showmanship.
        """)
    String chat(@UserMessage String userMessage);
}
```

💡 **WHAT'S HAPPENING**

- `@RegisterAIService` tells the CDI extension to create an injectable bean for this interface
- `chatModelName = "my-model"` references the model configuration in `microprofile-config.properties`
- `@SystemMessage` defines the AI's persona and behavior
- `@UserMessage` marks the parameter as user input

2 Wire the REST Endpoint

Open `ChatResource.java`. Inject the AI service and call it:

```
@Path("/chat")
@ApplicationScoped
public class ChatResource {

    @Inject
    ChatAssistant assistant;  // CDI injection!

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String chat(String message) {
        return assistant.chat(message);
    }
}
```

✓ KEY INSIGHT

The business code has **zero dependency on LangChain4j**. The only import is your own interface. Swap AI frameworks tomorrow? Change the implementation, not the business code.

3 Configure the Model

Open `src/main/resources/META-INF/microprofile-config.properties` and uncomment/add the model configuration:

```
# Chat model (Mistral AI)
dev.langchain4j.cdi.plugin.my-model.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.my-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-model.config.model-name=mistral-small-latest
dev.langchain4j.cdi.plugin.my-model.config.temperature=0.9
```

> **⚠ CRITICAL: THE .CONFIG. PREFIX**
>
> Builder properties **must** use the `.config.` prefix: `dev.langchain4j.cdi.plugin.<name>.config.<prop>=value`. Without `.config.`, the property is **silently ignored**.

Deploy and Test

```
mvn clean wildfly:dev
```

```
# In another terminal:
curl -X POST -H "Content-Type: text/plain" \
  -d "Tell me a joke about Java developers in Vegas" \
  http://localhost:8080/demo-1/api/chat
```

4 Add Streaming Responses

Open `ChatAssistantStreaming.java` and create a streaming AI service:

```
@RegisterAIService(streamingChatModelName = "my-streaming-model")
public interface ChatAssistantStreaming {

    @SystemMessage("You are a creative assistant for a Vegas show...")
    TokenStream chatStream(@UserMessage String userMessage);
}
```

Then in `ChatResource.java`, add the SSE endpoint:

```
@Inject
ChatAssistantStreaming streamingAssistant;

@GET
@Path("/stream")
@Produces(MediaType.SERVER_SENT_EVENTS)
public void chatStream(@QueryParam("message") String message,
                       @Context SseEventSink sink, @Context Sse sse) {
    TokenStream tokenStream = streamingAssistant.chatStream(message);

    tokenStream
        .onPartialResponse(token ->
            sink.send(sse.newEvent("token", token)))
        .onCompleteResponse(resp -> {
            sink.send(sse.newEvent("done", ""));
            sink.close();
        })
        .onError(error -> {
            sink.send(sse.newEvent("error", error.getMessage()));
            sink.close();
        })
        .start();
}
```

Add the streaming model config in `microprofile-config.properties`:

```
dev.langchain4j.cdi.plugin.my-streaming-model.class=\
    dev.langchain4j.model.mistralai.MistralAiStreamingChatModel
dev.langchain4j.cdi.plugin.my-streaming-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.my-streaming-model.config.model-name=mistral-small-latest
```

5 Vision: Image Analysis

Open `ImageAnalyzerServlet.java` and inject a vision model:

```
@WebServlet("/uploadServlet")
@MultipartConfig
public class ImageAnalyzerServlet extends HttpServlet {

    @Inject
    @Named("vision-model")
    ChatModel visionModel;

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) throws ... {
        Part file = request.getPart("file");

        UserMessage userMessage = UserMessage.from(
            TextContent.from("Describe this image in detail"),
            ImageContent.from(encodeBase64(file.getInputStream()),
                              file.getContentType())
        );

        ChatResponse answer = visionModel.chat(userMessage);
        response.getWriter().write(answer.aiMessage().text());
    }
}
```

And the vision model config:

```
dev.langchain4j.cdi.plugin.vision-model.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.vision-model.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.vision-model.config.model-name=pixtral-large-latest
```

✓ Test & Validate Exercise 1

```
# Regular chat
curl -X POST -H "Content-Type: text/plain" \
  -d "What is CDI in 2 sentences?" \
  http://localhost:8080/demo-1/api/chat

# Streaming (open in browser)
# http://localhost:8080/demo-1/stream.html

# Image analysis (open in browser)
# http://localhost:8080/demo-1/image.html

# Health check
curl http://localhost:8080/demo-1/api/chat/health
```

- ✓ Regular chat returns a response from the Vegas comedian
- ✓ Streaming shows tokens arriving in real-time
- ✓ Image upload returns an AI-generated description

> 💡 **STUCK?**
>
> Switch to the solution: `cd ../solution && mvn clean wildfly:dev`

Exercise 2: Fault Tolerance & Telemetry

Key message: "MicroProfile annotations work on AI Services because they are CDI beans"

> 📌 **GOAL**
>
> Start with a working booking assistant that already has Memory, RAG, and Tools. Your task: add MicroProfile Fault Tolerance annotations to make it production-ready, then observe it with OpenTelemetry.

```
cd demo-project/demo-2-ft-telemetry/base
mvn clean wildfly:dev
```

What's Already Working

The base module comes pre-wired with these features:

🧠

Conversation Memory

`@MemoryId` provides per-session memory via `ChatMemoryProviderBean`. Each browser tab gets its own conversation.

📄

RAG

`ShowDetailsProvider` ingests show data into an in-memory vector store using Mistral embeddings.

🔧

Tools

`BookingTools` is a CDI bean with 5 `@Tool` methods: list, register, cancel, check availability, view bookings.

Test that everything works before adding Fault Tolerance:

```
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-123" \
  -d "What shows are available?" \
  http://localhost:8080/demo-2/api/chat
```

1 Add @Retry

Open `ChatAssistant.java`. The AI service interface already has `@RegisterAIService` with memory, RAG, and tools. Add the `@Retry` annotation:

```
import org.eclipse.microprofile.faulttolerance.Retry;

@RegisterAIService(chatModelName = "my-model",
                   chatMemoryProviderName = "my-memory",
                   contentRetrieverName = "my-rag",
                   tools = BookingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    String chat(@MemoryId String sessionId,
                @UserMessage String message);
}
```

💡 **WHY THIS WORKS**

Because `@RegisterAIService` creates a CDI bean, **all MicroProfile interceptors apply**. The CDI container wraps the proxy with the Fault Tolerance interceptor — no glue code needed.

2 Add @Timeout and @Fallback

Stack the Fault Tolerance annotations:

```
import org.eclipse.microprofile.faulttolerance.*;
import java.time.temporal.ChronoUnit;

@RegisterAIService(chatModelName = "my-model",
                   chatMemoryProviderName = "my-memory",
                   contentRetrieverName = "my-rag",
                   tools = BookingTools.class)
public interface ChatAssistant {

    @Retry(maxRetries = 3, delay = 1000)
    @Timeout(value = 30, unit = ChronoUnit.SECONDS)
    @Fallback(fallbackMethod = "chatFallback")
    String chat(@MemoryId String sessionId,
                @UserMessage String message);

    default String chatFallback(String sessionId, String message) {
        return "I'm sorry, our AI service is temporarily unavailable. "
             + "Please try again in a moment. "
             + "Our team has been notified.";
    }
}
```

3 Add @CircuitBreaker

Add the final piece — the circuit breaker prevents flooding a failing service:

```
@Retry(maxRetries = 3, delay = 1000)
@Timeout(value = 30, unit = ChronoUnit.SECONDS)
@Fallback(fallbackMethod = "chatFallback")
@CircuitBreaker(requestVolumeThreshold = 5, failureRatio = 0.5)
String chat(@MemoryId String sessionId,
            @UserMessage String message);
```

Hot-reload: save and run `mvn package` (or let your IDE auto-build).

✓ TEST THE RESILIENCE

Simulate a failure: temporarily change the API key to an invalid value, then send requests. You should see retries, then the fallback message. Restore the key and the circuit breaker recovers automatically.

4 Add OpenTelemetry Observability

Start the Grafana LGTM stack (Loki + Grafana + Tempo + Mimir):

```
# Start the observability stack
podman run -p 3000:3000 -p 4317:4317 -p 4318:4318 \
           --rm -ti grafana/otel-lgtm
```

The telemetry configuration is already in `microprofile-config.properties` :

```
# OpenTelemetry
otel.exporter.otlp.endpoint=http://localhost:4318
otel.service.name=demo-2-langchain4j-cdi
otel.traces.exporter=otlp
otel.metrics.exporter=otlp

# LangChain4j-CDI Telemetry Listeners
dev.langchain4j.cdi.plugin.my-model.config.listeners=\
    dev.langchain4j.cdi.telemetry.SpanChatModelListener,\
    dev.langchain4j.cdi.telemetry.MetricsChatModelListener
```

Send a few chat requests, then open http://localhost:3000 (Grafana). Navigate to **Explore** → **Tempo** and search for service `demo-2-langchain4j-cdi` .

You will see distributed traces showing:

- LLM call latency
- Token counts (in/out)
- Tool calls and their durations
- RAG retrieval steps

✓ Test & Validate Exercise 2

```
# Chat with session
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "What shows are available?" \
  http://localhost:8080/demo-2/api/chat

# Register for a show
curl -X POST -H "Content-Type: text/plain" \
  -H "X-Session-Id: test-456" \
  -d "Register John Doe for the first show" \
  http://localhost:8080/demo-2/api/chat

# Inspect memory
curl "http://localhost:8080/demo-2/api/chat/memory?sessionId=test-456"
```

- ✓ Memory retains context across messages in the same session
- ✓ RAG retrieves relevant show information
- ✓ Tools execute bookings, cancellations, and queries
- ✓ Fallback responds gracefully when the AI is unavailable
- ✓ Traces appear in Grafana/Tempo

Exercise 3: MCP Integration

Key message: "MCP is JDBC for AI — your agents talk to any tool server"

> 📌 **GOAL**
>
> Connect an AI agent to an external MCP (Model Context Protocol) server. The MCP server provides dice-rolling tools, and your AI agent plays a casino craps game via the protocol.

What is MCP?

The **Model Context Protocol** is an open standard for connecting AI agents to external tools. Think of it as a universal adapter: one protocol, any tool server, any language.

```
🚀 Agent CDI    →    🔌 McpToolProvider    →    🌐 MCP Server (any language)
@RegisterAIService        toolProviderName          JSON-RPC via HTTP SSE
```

0 Build & Start the MCP Server

First, build and start the external MCP dice server:

```
# Build the MCP server
cd demo-project/demo-3-mcp/mcp-server
mvn clean package

# Start it (runs on port 8090)
java -jar target/mcp-server-1.0-SNAPSHOT-jar-with-dependencies.jar
```

The MCP server exposes a `roll` tool that generates random dice results. It communicates via HTTP SSE (Server-Sent Events) with JSON-RPC 2.0.

> 💡 **KEEP THIS RUNNING**
>
> Leave the MCP server running in a separate terminal. The AI agent will connect to it via HTTP SSE on `localhost:8090/sse`.

1 Create the Casino Dealer AI Service

Open `CasinoDealerAI.java` in `demo-3-mcp/base/` :

```
@RegisterAIService(chatModelName = "mistral",
                   toolProviderName = "mcp")
public interface CasinoDealerAI {

    @SystemMessage("""
        You are Lucky Jack Diamond, the charismatic dealer
        at "The Golden Ace Casino" in Las Vegas.
        You run the Craps dice game.

        RULES: Roll 2 dice. Use the roll tool with numberOfDice=2.
        - 7 or 11 on first roll = WIN
        - 2, 3, or 12 on first roll = LOSE (Craps)
        - Any other = that number is the "Point"

        REQUIRED FORMAT:
        DICE: [X, Y]
        TOTAL: [sum]
        RESULT: [what happened]
        """)
    String play(@UserMessage String playerAction);
}
```

💡 **TOOLPROVIDERNAME**

`toolProviderName = "mcp"` connects this agent to the `McpToolProvider` bean, which automatically discovers tools from the external MCP server.

2 Configure MCP via Properties

Open `microprofile-config.properties` and uncomment the MCP configuration:

```
# Chat Model (Mistral AI)
dev.langchain4j.cdi.plugin.mistral.class=\
    dev.langchain4j.model.mistralai.MistralAiChatModel
dev.langchain4j.cdi.plugin.mistral.config.api-key=${MISTRAL_API_KEY}
dev.langchain4j.cdi.plugin.mistral.config.model-name=mistral-small-latest

# MCP Transport (SSE to dice server)
dev.langchain4j.cdi.plugin.ssetransport.class=\
    dev.langchain4j.mcp.client.transport.http.HttpMcpTransport
dev.langchain4j.cdi.plugin.ssetransport.config.sseUrl=\
    http://localhost:8090/sse

# MCP Client
dev.langchain4j.cdi.plugin.mcpclient.class=\
    dev.langchain4j.mcp.client.DefaultMcpClient
dev.langchain4j.cdi.plugin.mcpclient.config.transport=lookup:@ssetransport

# MCP Tool Provider
dev.langchain4j.cdi.plugin.mcp.class=\
    dev.langchain4j.mcp.McpToolProvider
dev.langchain4j.cdi.plugin.mcp.config.mcpClients=lookup:@mcpclient
```

💡 **THE LOOKUP: PATTERN**

`lookup:@ssetransport` references another CDI bean by its config name. This is how you wire beans together purely through configuration — no Java code needed for the plumbing.

3 Wire the REST Endpoint

Open `GameResource.java`:

```
@Path("/game")
@ApplicationScoped
public class GameResource {

    @Inject
    CasinoDealerAI gameMaster;

    @POST
    @Path("/play")
    @Consumes(MediaType.TEXT_PLAIN)
    @Produces(MediaType.TEXT_PLAIN)
    public String play(String playerAction) {
        return gameMaster.play(playerAction);
    }

    @GET
    @Path("/start")
    @Produces(MediaType.TEXT_PLAIN)
    public String start() {
        return gameMaster.play(
            "Hello! I'm here to play Craps.");
    }
}
```

✓ Test & Validate Exercise 3

```
# Make sure the MCP server is running on port 8090!

# Deploy
cd demo-project/demo-3-mcp/base
mvn clean wildfly:dev

# Start a game
curl http://localhost:8080/demo-3/api/game/start

# Roll the dice
curl -X POST -H "Content-Type: text/plain" \
  -d "Roll the dice!" \
  http://localhost:8080/demo-3/api/game/play

# Open the web UI
# http://localhost:8080/demo-3/
```

- ✓ The AI agent calls the MCP server's `roll` tool via HTTP SSE
- ✓ Dice results are returned in the DICE: [X, Y] format
- ✓ Lucky Jack announces wins, losses, and points with Vegas flair
- ✓ The web UI renders visual dice with animated pips

🎲 **CONGRATULATIONS!**

You have built three AI-powered applications using LangChain4j-CDI: a simple chatbot, a resilient booking assistant, and an MCP-integrated casino game. All using standard Jakarta EE patterns.

What's Next

A2A: Agent-to-Agent Protocol

MCP connects agents to *tools*. **A2A** connects agents *to each other*. Imagine your Jakarta EE agent delegating tasks to other specialized agents, each running on its own server.

LangChain4j-CDI Modules

MODULE	ARTIFACT	PURPOSE
core	langchain4j-cdi-core	API + annotations
portable	langchain4j-cdi-portable-ext	WildFly, Payara, GlassFish, Liberty
build	langchain4j-cdi-build-compatible-ext	Quarkus, Helidon
config	langchain4j-cdi-config	MicroProfile Config SPI
FT	langchain4j-cdi-fault-tolerance	Retry, Timeout, CircuitBreaker
telemetry	langchain4j-cdi-telemetry	OpenTelemetry spans + metrics

Resources

- LangChain4j-CDI on GitHub
- LangChain4j Documentation
- MCP Specification
- WildFly Documentation

Troubleshooting

PROBLEM	CAUSE	SOLUTION
`ClassNotFoundException: ...faulttolerance.Retry`	Missing Galleon layers	Add `microprofile-fault-tolerance` in `<layers>`
Properties silently ignored	Missing `.config.` prefix	Use `...plugin.X.config.prop=val`
Memory resets every message	`get()` creates new instance	Use `computeIfAbsent()` in provider
`ChatMessage.text()` won't compile	`ChatMessage` has no `text()`	Pattern-match: `SystemMessage`, `UserMessage`, etc.
404 on context root	No `<name>` in wildfly plugin	Add `<name>demo-N</name>`
MCP server not responding	Server not running on 8090	Start the MCP server first
`MISTRAL_API_KEY` not found	Environment variable not set	`export MISTRAL_API_KEY=your-key`

⚠ EMERGENCY BACKUP

If anything goes wrong, switch to the solution module:

```
cd demo-N-xxx/solution && mvn clean wildfly:dev
```


By **Yann Blazart** & **Emmanuel Hugonnet**